

Experiment: 8

Student Name: Sahil

UID:24BDA70148

Branch: CSE

Section/Group:AIT-KRG-GP2

Semester: 4th

DATE: 31/03/2026

Subject Name: DBMS

1. Aim of the Practical

To create and execute a stored procedure that retrieves employee data based on gender using input parameters, demonstrating the use of procedures, functions, IN/OUT parameters, and procedural logic in PostgreSQL.

2. Tool Used

Database Management System: PostgreSQL

Database Administration Tool: pgAdmin

3. Objective

To write and execute a PL/SQL stored procedure that dynamically accepts gender as an argument and computes the employee count corresponding to the given gender. Also to create a function that returns a table of employee data.

4. Practical / Experimental Steps

- Step 1: Create the base table employees with attributes emp_id, emp_name, gender, and salary.
- Step 2: Insert sample records into the employees table to simulate real-world employee data.
- Step 3: Define and create a stored procedure get_Employee_Count_BY_Gender that accepts gender as an IN parameter, returns count as an OUT parameter, and uses an INOUT status parameter.
- Step 4: Implement logic inside the procedure to count employees matching the given gender using a SELECT COUNT(*) query.
- Step 5: Execute the stored procedure using a DO block with dynamically declared variables.
- Step 6: Display the output using RAISE NOTICE to show gender, employee count, and status.
- Step 7: Create a function Get_Data() that returns a table of employee names and salaries using RETURN QUERY.
- Step 8: Execute the function using SELECT * FROM Get_Data() and verify the output.

5. I/O Analysis

5.1 Create Table

```
CREATE TABLE employees (
    emp_id SERIAL PRIMARY KEY,
    emp_name VARCHAR(50),
    gender VARCHAR(10),
    salary NUMERIC(10,2)
);
```

```
temporary=# CREATE TABLE employees (
temporary#     emp_id SERIAL PRIMARY KEY,
temporary#     emp_name VARCHAR(50),
temporary#     gender VARCHAR(10),
temporary#     salary NUMERIC(10,2)
temporary# );
CREATE TABLE
```

5.2 Insert Sample Data

```
INSERT INTO employees (emp_name, gender, salary) VALUES
('Amit', 'Male', 30000),
('Riya', 'Female', 35000),
('John', 'Male', 28000),
('Sneha', 'Female', 40000);
```

```
temporary=# INSERT INTO employees (emp_name, gender, salary) VAL
temporary# ('Amit', 'Male', 30000),
temporary# ('Riya', 'Female', 35000),
temporary# ('John', 'Male', 28000),
temporary# ('Sneha', 'Female', 40000)
temporary# ;
INSERT 0 4
```

5.3 Stored Procedure - get_Employee_Count_BY_Gender

```
CREATE OR REPLACE PROCEDURE get_Employee_Count_BY_Gender(
    IN P_GEN VARCHAR(20),
    OUT COUNT_EMP INT,
    INOUT STATUS VARCHAR
)
AS
$$
BEGIN
    SELECT COUNT(*) INTO COUNT_EMP
    FROM EMPLOYEES
    WHERE GENDER = P_GEN;
    STATUS := 'SUCCESS';
END;
$$ LANGUAGE PLPGSQL;
```

```
temporary=# CREATE OR REPLACE PROCEDURE get_Employee_Count_BY_Gender(IN P_GEN VARCHAR(20), OUT COUNT_EMP INT, INOUT STATUS VARC
temporary# AS
temporary# $$
temporary# BEGIN
temporary# SELECT COUNT(*) INTO COUNT_EMP FROM EMPLOYEES WHERE GENDER=P_GEN;
temporary# STATUS:='SUCCESS';
temporary# END;
temporary# $$ LANGUAGE PLPGSQL;
CREATE PROCEDURE
```

5.4 Execute Procedure using DO Block

```
DO
$$
DECLARE
    GEN VARCHAR(20) := 'Male';
    COUNT_BY_GEN INT;
    STATUS VARCHAR(20) := 'Fail';
BEGIN
    CALL get_Employee_Count_BY_Gender(GEN, COUNT_BY_GEN, STATUS);
    RAISE NOTICE 'GENDER IS % YOUR COUNT IS % AND STATUS IS %',
        GEN, COUNT_BY_GEN, STATUS;
END;
$$;
```

Expected Output: NOTICE: GENDER IS Male YOUR COUNT IS 2 AND STATUS IS SUCCESS

```
temporary=# DO
temporary=# $$
temporary$# DECLARE
temporary$#     GEN VARCHAR(20):='Male';
temporary$#     COUNT_BY_GEN INT;
temporary$#     STATUS VARCHAR(20):='Fail';
temporary$# BEGIN
temporary$#
temporary$#     CALL get_Employee_Count_BY_Gender(GEN,COUNT_BY_GEN,STATUS);
temporary$#
temporary$#     RAISE NOTICE 'GENDER IS % YOUR COUNT IS % AND STATUS IS %',GEN,COUNT_BY_GEN,STATUS;
temporary$#
temporary$# END;
temporary$# $$;
NOTICE:  GENDER IS Male YOUR COUNT IS 2 AND STATUS IS SUCCESS
```

5.5 Function - Get_Data()

```
CREATE OR REPLACE FUNCTION Get_Data()
RETURNS TABLE(name_emp VARCHAR(10), emp_salary NUMERIC(10,2))
AS
$$
BEGIN
    RETURN QUERY SELECT emp_name, salary FROM employees;
END;
$$ LANGUAGE PLPGSQL;
```

```
SELECT * FROM Get_Data();
```

```
temporary=# SELECT * FROM Get_Data();
name_emp | emp_salary
-----+-----
Amit     | 30000.00
Riya     | 35000.00
John     | 28000.00
Sneha    | 40000.00
(4 rows)
```

Expected Output:

Returns a table with columns name_emp and emp_salary listing all employee records.

6. Learning Outcomes

- Understood the structure and syntax of stored procedures in PostgreSQL using PL/pgSQL.
- Learned how to use IN, OUT, and INOUT parameters in procedures.
- Practiced passing dynamic input values to procedures using a DO block and CALL statement.
- Applied aggregate functions like COUNT(*) inside procedural logic for data analysis.
- Learned how to create functions that return tables using RETURNS TABLE and RETURN QUERY.
- Developed reusable database logic that can be applied in real-world business scenarios.